Symbiosis Institute of Technology, Pune

Faculty of Engineering
CSE- Academic Year 2026-27
Java Programming – Lab Batch 2024-28

Lab Assignment No:- 25	
Name of Student	Abishek Yadav
PRN No.	25070122014
Batch	A1
Class	CSE
Academic Year & Semester	2026-2027/3RD
Date of Submission	15/10/2026
Title of Assignment:	Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.
Theory:	A thread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the run() method. A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as start(), sleep(), join(), and interrupt() control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications.
Practice Questions	1. Create a program where multiple threads print numbers simultaneously.

CODE:

```java
class NumberThread extends Thread {
    private int start;
    private int end;

    NumberThread(String name, int start, int end) {
        super(name);
        this.start = start;
        this.end = end;
    }

    public void run() {
        System.out.println(getName() + " started.");
        for (int i = start; i <= end; i++) {
            System.out.println(getName() + " : " + i);
            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                System.out.println(getName() + " interrupted.");
            }
        }
        System.out.println(getName() + " terminated.");
    }
}

public class exp25 {
    public static void main(String[] args) {
        Thread t1 = new NumberThread("Thread 1", 1, 5);
        Thread t2 = new NumberThread("Thread 2", 6, 10);
        Thread t3 = new NumberThread("Thread 3", 11, 15);

        System.out.println("Main thread started.");

        t1.start();
        t2.start();
        t3.start();

        try {
            t1.join();
            t2.join();
```

```
            t3.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }

        System.out.println("Main thread terminated.");
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>java exp25
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Main thread started.
Thread 1 started.
Thread 2 started.
Thread 3 started.
Thread 2 : 6
Thread 3 : 11
Thread 1 : 1
Thread 3 : 12
Thread 2 : 7
Thread 1 : 2
Thread 2 : 8
Thread 3 : 13
Thread 1 : 3
Thread 2 : 9
Thread 1 : 4
Thread 3 : 14
Thread 1 : 5
Thread 2 : 10
Thread 3 : 15
Thread 1 terminated.
Thread 2 terminated.
Thread 3 terminated.
Main thread terminated.


C:\Users\Bina Computer\OneDrive\Documents\Java>
```

QUESTION

Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

CODE:

```
class BankAccount {
    private int balance = 1000;

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println(
            Thread.currentThread().getName()
            + " deposited Rs." + amount
            + " | Balance = Rs." + balance
        );
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println(
                Thread.currentThread().getName()
                + " withdrew Rs." + amount
                + " | Balance = Rs." + balance
            );
        } else {
            System.out.println(
                Thread.currentThread().getName()
                + " tried to withdraw Rs." + amount
                + " | Insufficient Balance"
            );
        }
    }

    int getBalance() {
        return balance;
    }
}

class DepositThread extends Thread {
    private BankAccount account;

    DepositThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        int[] deposits = {500, 700, 300};
        for (int amount : deposits) {
```

```
            account.deposit(amount);
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                System.out.println("Deposit thread interrupted.");
            }
        }
    }
}

class WithdrawalThread extends Thread {
    private BankAccount account;

    WithdrawalThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        int[] withdrawals = {400, 600, 500};
        for (int amount : withdrawals) {
            account.withdraw(amount);
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                System.out.println("Withdrawal thread
interrupted.");
            }
        }
    }
}

public class exp25_2 {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Thread depositThread = new DepositThread(account);
        Thread withdrawalThread = new WithdrawalThread(account);

        depositThread.setName("Deposit Thread");
        withdrawalThread.setName("Withdrawal Thread");

        System.out.println("Bank transaction started.");
        System.out.println("Initial Balance = Rs." +
account.getBalance());

        depositThread.start();
        withdrawalThread.start();

        try {
            depositThread.join();
            withdrawalThread.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }

        System.out.println("Final Balance = Rs." +
account.getBalance());
        System.out.println("Bank transaction completed.");
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>java exp25_2
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Bank transaction started.
Initial Balance = Rs.1000
Deposit Thread deposited Rs.500 | Balance = Rs.1500
Withdrawal Thread withdrew Rs.400 | Balance = Rs.1100
Deposit Thread deposited Rs.700 | Balance = Rs.1800
Withdrawal Thread withdrew Rs.600 | Balance = Rs.1200
Deposit Thread deposited Rs.300 | Balance = Rs.1500
Withdrawal Thread withdrew Rs.500 | Balance = Rs.1000
Final Balance = Rs.1000
Bank transaction completed.


C:\Users\Bina Computer\OneDrive\Documents\Java>
```

Conclusion	The programs successfully demonstrate multithreading using the Thread class. Multiple threads can execute tasks concurrently. The first program demonstrates simultaneous number printing along with the thread lifecycle (New, Runnable, Timed Waiting, Terminated), while the second demonstrates concurrent bank deposit and withdrawal operations using synchronization to maintain a consistent balance.